

DOCUMENTO DE ARQUITECTURA DE SOLUCIÓN

Microservicio de Autenticación — Auth Service

Proyecto	Auth Service v1.0
Stack	Python • FastAPI • PostgreSQL • JWT
Clasificación	Confidencial
Versión Doc	1.0.0
Fecha	5 de mayo de 2026
Autor	Equipo de Arquitectura de Soluciones

Índice

1. Resumen Ejecutivo	4
1.1. Objetivos Estratégicos	4
1.2. Alcance	4
2. Partes Interesadas	5
3. Requerimientos	6
3.1. Requerimientos Funcionales	6
3.2. Requerimientos No Funcionales	6
4. Casos de Uso	8
4.1. CU-01 — Registrar Usuario	8
4.1.1. Flujo Principal	8
4.1.2. Flujos Alternativos / Excepciones	8
4.2. CU-02 — Iniciar Sesión (Login)	8
4.2.1. Flujo Principal	8
4.2.2. Flujos Alternativos / Excepciones	9
4.3. CU-03 — Renovar Access Token (Refresh)	9
4.3.1. Flujo Principal	9
4.3.2. Flujos Alternativos / Excepciones	9
4.4. CU-04 — Solicitar Recuperación de Contraseña	9
4.4.1. Flujo Principal	10
4.4.2. Flujos Alternativos / Excepciones	10
4.5. CU-05 — Restablecer Contraseña	10
4.5.1. Flujo Principal	10
4.5.2. Flujos Alternativos / Excepciones	10
4.6. CU-06 — Consultar Perfil de Usuario	10
4.6.1. Flujo Principal	11
4.6.2. Flujos Alternativos / Excepciones	11
5. Matriz de Trazabilidad	12
5.1. Casos de Uso vs. Requerimientos Funcionales	12
5.2. Cobertura de Requerimientos No Funcionales	12
6. Arquitectura de Solución	13
6.1. Vista de Contexto	13
6.2. Componentes Internos del Servicio	13
6.3. Modelo de Datos	14
7. Diagramas de Actividad	15
7.1. Flujo: Registrar Usuario (CU-01)	15
7.2. Flujo: Iniciar Sesión — Login (CU-02)	16
7.3. Flujo: Renovar Token — Refresh (CU-03)	17
7.4. Flujo: Recuperación de Contraseña (CU-04 y CU-05)	17
8. Consideraciones de Seguridad	19
8.1. OWASP Top 10 — Controles Implementados	19
8.2. Gestión de Secretos	19

9. Estrategia de Pruebas	20
9.1. Tipos de Prueba	20
9.2. Casos de Prueba Clave	20
10. Estructura del Proyecto y Despliegue	22
10.1. Estructura de Archivos	22
10.2. Variables de Entorno Requeridas	22
11. Prompt de Ingeniería para Generación de Código con IA	23
12. Comparación de Resultados entre IAs	24
12.1. Edge Cases Identificados por IA	24
13. Glosario	26

1 Resumen Ejecutivo

El presente documento describe la arquitectura de solución para el **Auth Service**, un microservicio independiente de autenticación y gestión de identidad diseñado bajo los principios de arquitectura de microservicios, seguridad por diseño (*Security by Design*) y alta disponibilidad.

El servicio expone una API RESTful construida con **FastAPI** sobre **Python 3.11+**, respaldada por **PostgreSQL 15** como base de datos relacional, y utiliza tokens **JWT** (JSON Web Tokens) con esquema de rotación de refresh tokens para garantizar sesiones seguras y escalables.

1.1 Objetivos Estratégicos

- Centralizar la gestión de identidad en un único servicio reutilizable por múltiples aplicaciones del ecosistema.
- Garantizar seguridad end-to-end: contraseñas hasheadas con bcrypt (cost ≥ 12), tokens con tiempo de expiración controlado y rotación automática.
- Permitir escalabilidad horizontal independiente del resto de la plataforma.
- Reducir la superficie de ataque mediante separación de responsabilidades (SRP) y principio de mínimo privilegio.
- Proveer trazabilidad y auditoría de todas las operaciones de autenticación.

1.2 Alcance

- Registro de usuarios con validación de identidad.
- Autenticación mediante credenciales y emisión de tokens JWT (access + refresh).
- Renovación de sesión mediante refresh token con rotación.
- Recuperación de contraseña mediante token de un solo uso enviado por email.
- Consulta del perfil del usuario autenticado.

Fuera del alcance: autorización basada en roles (RBAC), gestión de permisos, autenticación OAuth/SAML, integración con proveedores externos (Google, GitHub).

2 Partes Interesadas

Rol	Nombre / Equipo	Interés	Nivel de Influencia
Arquitecto de Soluciones	Equipo de Arquitectura	Diseño técnico y cumplimiento de estándares	Alto
Desarrollador Backend	Equipo Backend	Implementación del servicio	Alto
Oficial de Seguridad	InfoSec	Cumplimiento de políticas de seguridad	Alto
Product Owner	Negocio	Entrega de funcionalidades en tiempo	Medio
DevOps / SRE	Infraestructura	Despliegue, monitoreo y disponibilidad	Medio
QA Engineer	Calidad	Cobertura de pruebas y calidad del código	Medio

3 Requerimientos

3.1 Requerimientos Funcionales

ID	Nombre	Descripción	Prioridad
RF-01	Registro de usuario	El sistema debe permitir registrar un usuario con email, nombre completo y contraseña.	Alta
RF-02	Validación de email único	El sistema debe rechazar registros con un email ya existente.	Alta
RF-03	Hash de contraseña	Las contraseñas deben almacenarse hasheadas con bcrypt (cost factor ≥ 12).	Alta
RF-04	Login con JWT	El sistema debe autenticar usuarios y emitir access token (15 min) y refresh token (7 días).	Alta
RF-05	Renovación de sesión	El sistema debe permitir renovar el access token usando un refresh token válido y rotarlo.	Alta
RF-06	Recuperación de contraseña	El sistema debe enviar un token de un solo uso al email registrado para restablecer contraseña.	Alta
RF-07	Respuesta neutra en recuperación	El endpoint de recuperación siempre responde 200 OK sin revelar si el email existe.	Alta
RF-08	Consulta de perfil	El sistema debe retornar los datos del usuario autenticado dado un Bearer token válido.	Media
RF-09	Bloqueo por intentos fallidos	El sistema debe bloquear la cuenta 15 minutos tras 5 intentos fallidos consecutivos.	Alta
RF-10	Normalización de email	El email debe normalizarse a minúsculas antes de almacenarse.	Media

3.2 Requerimientos No Funcionales

ID	Nombre	Descripción	Prioridad
RNF-01	Disponibilidad	El servicio debe tener disponibilidad $\geq 99.9\%$ mensual.	Alta
RNF-02	Rendimiento	El p95 de latencia en endpoints de auth debe ser < 200 ms.	Alta
RNF-03	Escalabilidad	El servicio debe soportar escalado horizontal stateless (sin sesión en memoria).	Alta

ID	Nombre	Descripción	Prioridad
RNF-04	Seguridad en tránsito	Toda comunicación debe realizarse sobre HTTPS (TLS 1.2+).	Alta
RNF-05	Rate Limiting	Máximo 10 solicitudes por minuto por IP en endpoints de autenticación.	Alta
RNF-06	Auditoría	Todos los eventos de autenticación (éxito/fallo) deben registrarse con timestamp, IP y user-agent.	Media
RNF-07	Portabilidad	El servicio debe desplegarse como contenedor Docker.	Alta
RNF-08	Mantenibilidad	Cobertura de pruebas unitarias $\geq 80\%$.	Media
RNF-09	Conformidad OWASP	El servicio debe cumplir con OWASP Top 10 (2021), especialmente A02, A07.	Alta
RNF-10	Tiempo de expiración configurable	Los tiempos de expiración de tokens deben ser configurables por variables de entorno.	Baja

4 Casos de Uso

4.1 CU-01 — Registrar Usuario

Atributo	Detalle
ID	CU-01
Actor Principal	Usuario Anónimo
Prioridad	Alta
Precondición	El email no existe en el sistema.
Postcondición	Usuario creado en BD con contraseña hasheada.
Req. Relacionados	RF-01, RF-02, RF-03, RNF-09

4.1.1 Flujo Principal

1. El usuario envía email, nombre y contraseña.
2. El sistema valida formato de email (RFC 5322).
3. El sistema verifica que el email no esté registrado.
4. El sistema valida la contraseña contra la política definida.
5. El sistema hashea la contraseña con bcrypt.
6. El sistema almacena el usuario con estado activo.
7. El sistema retorna 201 Created con el perfil del usuario.

4.1.2 Flujos Alternativos / Excepciones

- **3a.** Email duplicado → 400 Bad Request con mensaje descriptivo.
- **4a.** Contraseña débil → 422 Unprocessable Entity.

4.2 CU-02 — Iniciar Sesión (Login)

Atributo	Detalle
ID	CU-02
Actor Principal	Usuario Registrado
Prioridad	Alta
Precondición	El usuario existe y su cuenta está activa.
Postcondición	Se emite par de tokens access + refresh.
Req. Relacionados	RF-04, RF-09, RF-10, RNF-02, RNF-05

4.2.1 Flujo Principal

1. El usuario envía email y contraseña.
2. El sistema normaliza el email a minúsculas.
3. El sistema busca el usuario por email.
4. El sistema verifica el hash de contraseña con bcrypt.
5. El sistema verifica que la cuenta no esté bloqueada.
6. El sistema genera access token (15 min) y refresh token (7 días).

7. El sistema almacena el hash del refresh token en BD.
8. El sistema retorna 200 OK con ambos tokens.

4.2.2 Flujos Alternativos / Excepciones

- **3a.** Usuario no encontrado → 401 Unauthorized.
- **4a.** Contraseña incorrecta → contador de intentos +1; si ≥ 5 → bloqueo 15 min.
- **5a.** Cuenta bloqueada → 403 Forbidden con tiempo restante.

4.3 CU-03 — Renovar Access Token (Refresh)

Atributo	Detalle
ID	CU-03
Actor Principal	Usuario Autenticado
Prioridad	Alta
Precondición	El usuario posee un refresh token válido y no expirado.
Postcondición	Nuevo par de tokens emitido; refresh token anterior invalidado.
Req. Relacionados	RF-05, RNF-03

4.3.1 Flujo Principal

1. El usuario envía el refresh token en el cuerpo de la solicitud.
2. El sistema verifica la firma y expiración del token JWT.
3. El sistema busca el hash del token en BD y verifica coincidencia.
4. El sistema invalida el refresh token anterior (rotación).
5. El sistema genera nuevo access token y nuevo refresh token.
6. El sistema retorna 200 OK con los nuevos tokens.

4.3.2 Flujos Alternativos / Excepciones

- **2a.** Token expirado → 401 Unauthorized.
- **3a.** Token no encontrado o ya usado → 401 Unauthorized (posible reutilización malintencionada).

4.4 CU-04 — Solicitar Recuperación de Contraseña

Atributo	Detalle
ID	CU-04
Actor Principal	Usuario Anónimo
Prioridad	Alta
Precondición	Ninguna.
Postcondición	Token de recuperación generado y email enviado (si el usuario existe).

Req. Relacionados	RF-06, RF-07, RNF-09
-------------------	----------------------

4.4.1 Flujo Principal

1. El usuario envía el email al endpoint de recuperación.
2. El sistema siempre responde 200 OK (seguridad: no revela existencia).
3. En segundo plano: si el email existe, el sistema genera un token SHA-256 de un solo uso.
4. El sistema almacena el token hashado con expiración de 30 minutos.
5. El sistema envía el token al email del usuario.

4.4.2 Flujos Alternativos / Excepciones

- **3a.** Email no existe → el proceso interno se detiene silenciosamente.

4.5 CU-05 — Restablecer Contraseña

Atributo	Detalle
ID	CU-05
Actor Principal	Usuario Anónimo
Prioridad	Alta
Precondición	El usuario posee el token de recuperación recibido por email.
Postcondición	La contraseña del usuario es actualizada y el token invalidado.
Req. Relacionados	RF-06, RF-07, RNF-09

4.5.1 Flujo Principal

1. El usuario envía el token y la nueva contraseña.
2. El sistema valida que el token exista en BD y no haya expirado.
3. El sistema valida la nueva contraseña contra la política.
4. El sistema hasha la nueva contraseña con bcrypt.
5. El sistema actualiza la contraseña y elimina el token.
6. El sistema retorna 200 OK.

4.5.2 Flujos Alternativos / Excepciones

- **2a.** Token inválido o expirado → 400 Bad Request.
- **3a.** Contraseña débil → 422 Unprocessable Entity.

4.6 CU-06 — Consultar Perfil de Usuario

Atributo	Detalle
ID	CU-06

Actor Principal	Usuario Autenticado
Prioridad	Media
Precondición	El usuario posee un access token válido.
Postcondición	Se retorna el perfil del usuario.
Req. Relacionados	RF-08, RNF-02

4.6.1 Flujo Principal

1. El usuario envía la solicitud con el Bearer token en cabecera Authorization.
2. El sistema valida la firma y expiración del access token.
3. El sistema extrae el user_id del payload del token.
4. El sistema consulta el usuario en BD.
5. El sistema retorna 200 OK con id, email, nombre y fecha de creación.

4.6.2 Flujos Alternativos / Excepciones

- **2a.** Token inválido o expirado → 401 Unauthorized.

5 Matriz de Trazabilidad

5.1 Casos de Uso vs. Requerimientos Funcionales

La siguiente matriz muestra qué requerimientos funcionales son satisfechos por cada caso de uso. La marca ✓ indica relación directa.

CU / RF	RF-01	RF-02	RF-03	RF-04	RF-05	RF-06	RF-07	RF-08	RF-09	RF-10
CU-01	✓	✓	✓	—	—	—	—	—	—	—
CU-02	—	—	—	✓	—	—	—	—	✓	✓
CU-03	—	—	—	—	✓	—	—	—	—	—
CU-04	—	—	—	—	—	✓	✓	—	—	—
CU-05	—	—	—	—	—	✓	✓	—	—	—
CU-06	—	—	—	—	—	—	—	✓	—	—

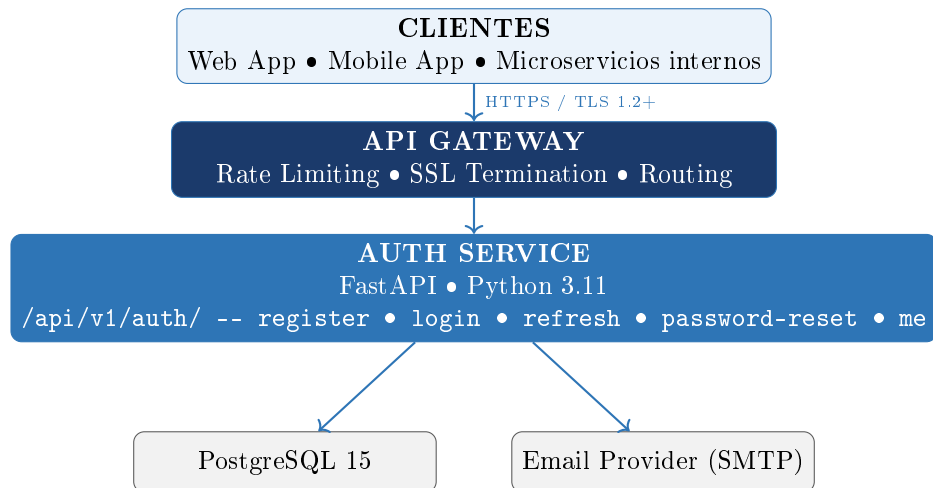
5.2 Cobertura de Requerimientos No Funcionales

ID	Nombre	Capa de implementación	Estado
RNF-01	Disponibilidad	Despliegue con Docker + health-check endpoint + múltiples réplicas	Diseño
RNF-02	Rendimiento	FastAPI async + connection pooling (SQLAlchemy) + índice en email	Diseño
RNF-03	Escalabilidad	Stateless: tokens JWT, sin sesión en memoria del servicio	Diseño
RNF-04	Seguridad TLS	Ingress / reverse proxy (nginx) con certificado TLS 1.2+	Infraestructura
RNF-05	Rate Limiting	Middleware SlowAPI (límite por IP configurado en variables de entorno)	Código
RNF-06	Auditoría	Middleware de logging estructurado (structlog) → ELK/CloudWatch	Código
RNF-07	Portabilidad	Dockerfile multi-stage + docker-compose para entorno local	Infraestructura
RNF-08	Cobertura pruebas	pytest + pytest-cov, umbral $\geq 80\%$ en CI/CD	Pruebas
RNF-09	OWASP	bcrypt, respuestas neutras, tokens de un solo uso, rate limiting	Código
RNF-10	Config. tokens	Todas las duraciones vía .env / variables de entorno (Pydantic Settings)	Código

6 Arquitectura de Solución

6.1 Vista de Contexto

El Auth Service opera como un microservicio autónomo que expone una API REST. Los clientes (aplicaciones web, móviles o microservicios internos) se comunican exclusivamente a través del API Gateway. El servicio gestiona su propio almacén de datos (PostgreSQL) y delega el envío de emails a un servicio externo (SMTP / Email Provider).



6.2 Componentes Internos del Servicio

Capa	Componente	Responsabilidad
API Layer	auth.py (Router)	Expone los endpoints REST, valida esquemas con Pydantic, delega a la capa de servicio.
Service Layer	auth_service.py	Orquesta la lógica de negocio: hashing, generación de tokens, validaciones.
Security Layer	security.py	Funciones de JWT (crear, verificar), hashing de contraseñas, generación de tokens seguros.
Data Layer	models/user.py	Modelo SQLAlchemy de la entidad User con todos sus campos y restricciones.
Schema Layer	schemas/auth.py	Modelos Pydantic para request/response con validaciones declarativas.
Config Layer	core/config.py	Pydantic Settings: carga variables de entorno con valores por defecto tipados.
DB Layer	db/session.py	Pool de conexiones SQLAlchemy, manejo de sesiones con context manager.
Migration Layer	alembic/	Migraciones versionadas de esquema de base de datos.

6.3 Modelo de Datos

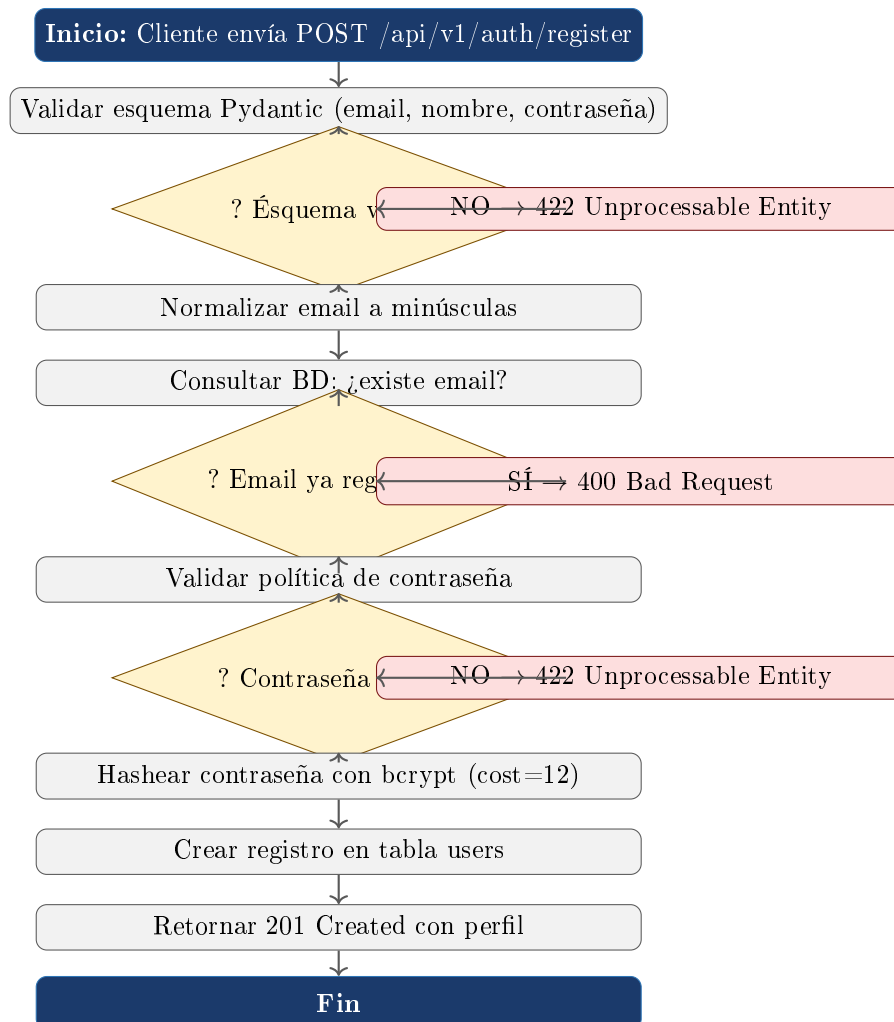
Tabla principal: **users** (PostgreSQL 15)

Campo	Tipo	PK/FK/UI	Restricciones y descripción
id	UUID	PK	Generado con uuid4(), no secuencial para evitar enumeración.
email	VARCHAR(255)	UK, NOT NULL	Normalizado a minúsculas. Indexado. Validado con regex RFC 5322.
hashed_password	TEXT	NOT NULL	Hash bcrypt con cost factor configurable (≥ 12 en producción).
full_name	VARCHAR(100)	NOT NULL	Nombre completo del usuario.
is_active	BOOLEAN	NOT NULL	DEFAULT true. false = cuenta deshabilitada administrativamente.
failed_attempts	INTEGER	NOT NULL	DEFAULT 0. Contador de intentos fallidos consecutivos.
locked_until	TIMESTAMP	NULLABLE	UTC. Nulo si la cuenta no está bloqueada.
reset_token	TEXT	NULLABLE	Hash SHA-256 del token de recuperación de contraseña.
reset_token_exp	TIMESTAMP	NULLABLE	UTC. Expiración del token (30 min desde creación).
created_at	TIMESTAMP	NOT NULL	DEFAULT now() UTC. No modificable.
updated_at	TIMESTAMP	NOT NULL	Actualizado automáticamente en cada modificación (trigger).

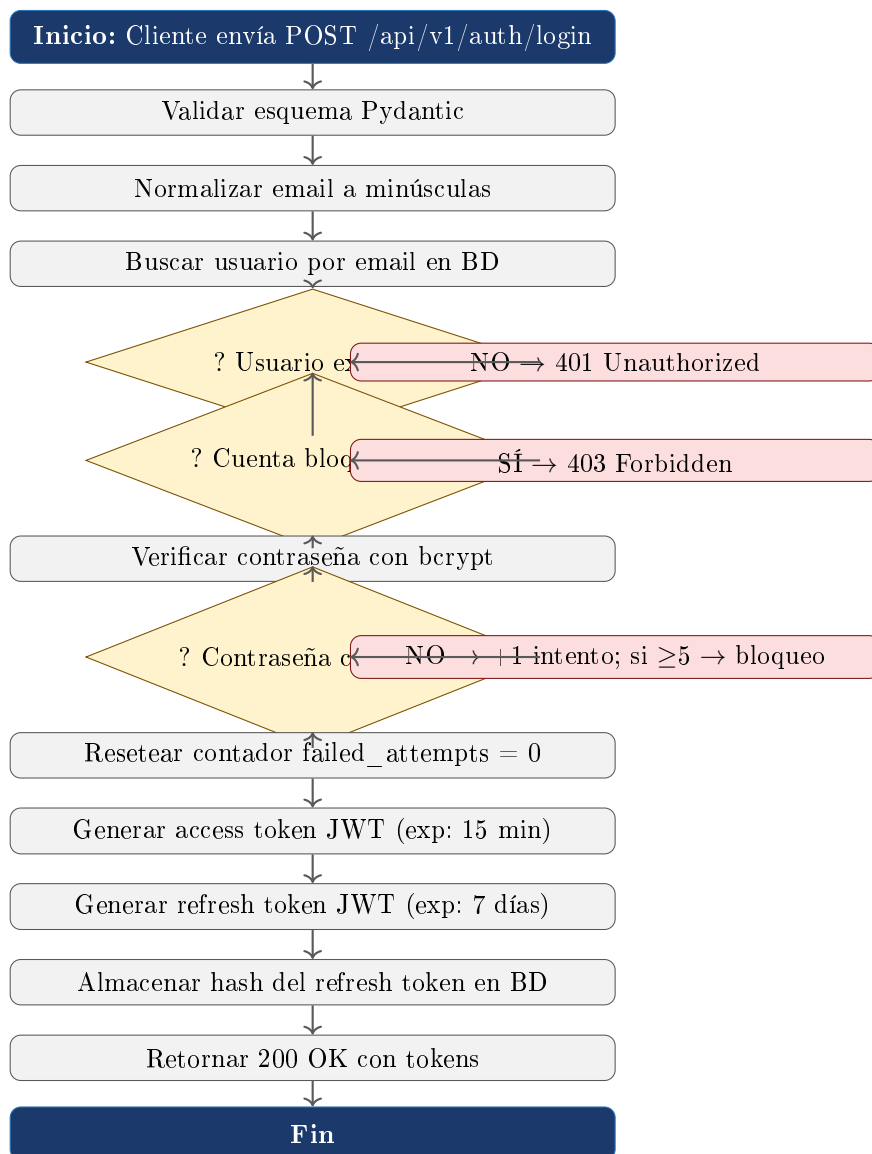
7 Diagramas de Actividad

Los siguientes diagramas describen el flujo de ejecución de los casos de uso principales. La notación utiliza: ● Inicio, ○ Fin, ◆ Decisión, → Flujo alternativo.

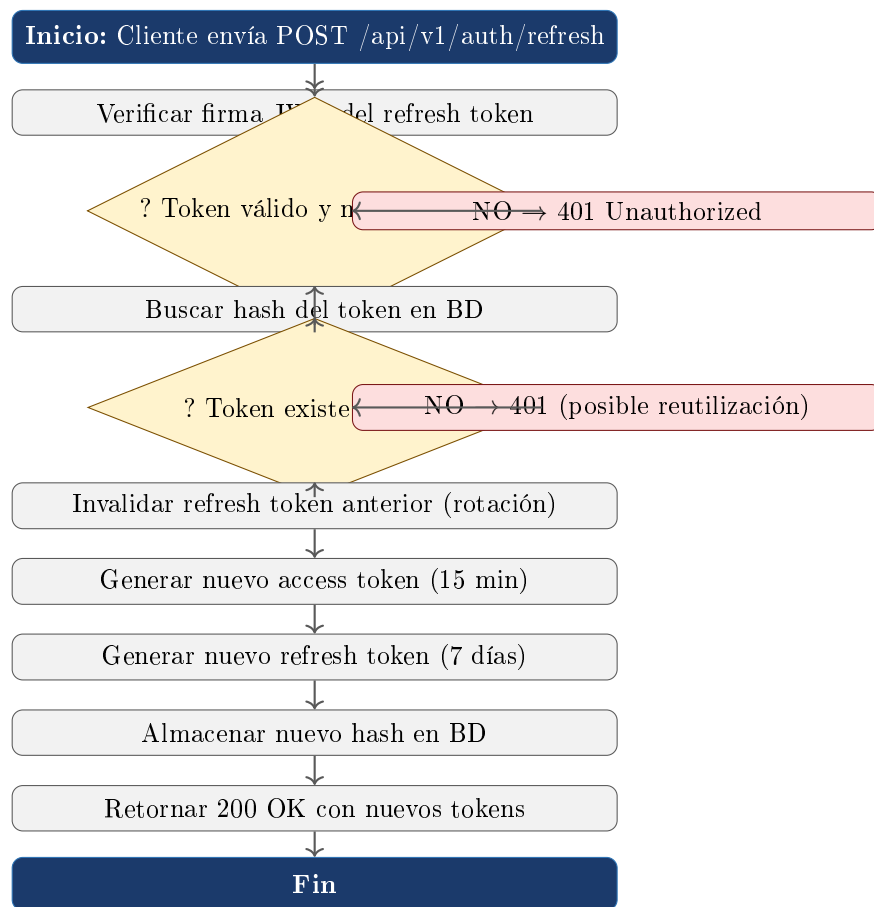
7.1 Flujo: Registrar Usuario (CU-01)



7.2 Flujo: Iniciar Sesión — Login (CU-02)

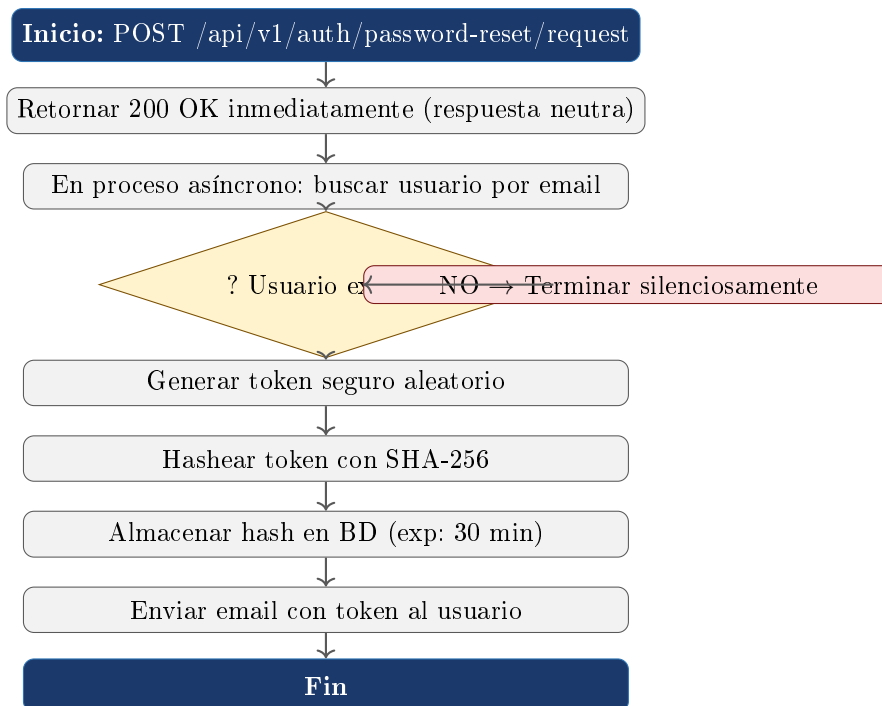


7.3 Flujo: Renovar Token — Refresh (CU-03)

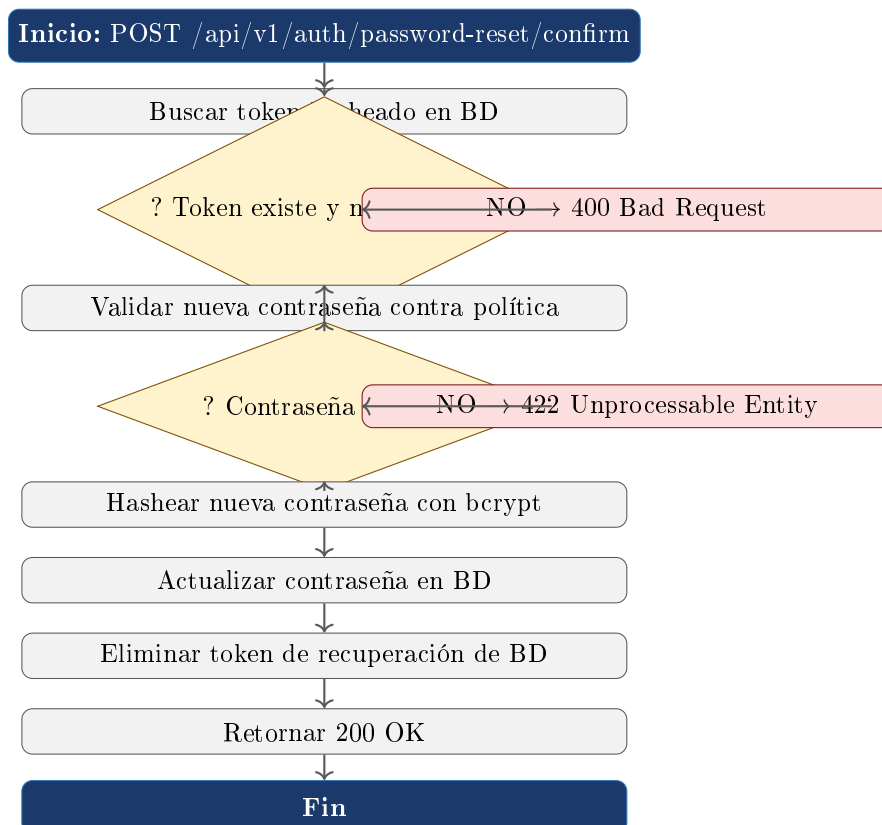


7.4 Flujo: Recuperación de Contraseña (CU-04 y CU-05)

Solicitud de recuperación:



Confirmación de restablecimiento:



8 Consideraciones de Seguridad

8.1 OWASP Top 10 — Controles Implementados

OWASP	Categoría	Control implementado
A01	Control de Acceso Roto	Tokens JWT firmados con HS256; validación de firma en cada request; refresh token con rotación.
A02	Fallos Criptográficos	bcrypt cost $\geq$ 12 para contraseñas; tokens de recuperación via secrets.token_urlsafes; HTTPS obligatorio.
A03	Inyección	Consultas con ORM SQLAlchemy (prepared statements); validación estricta con Pydantic.
A05	Configuración Incorrecta	Variables de entorno para todos los secretos; no se exponen trazas de error al cliente.
A07	Autenticación Rota	Rate limiting, bloqueo por intentos, tokens de corta vida, respuesta neutra en recuperación.
A09	Logging Insuficiente	Registro estructurado de todos los eventos de autenticación con timestamp e IP.

8.2 Gestión de Secretos

- **SECRET_KEY:** Clave de firma JWT — mínimo 256 bits — almacenada en gestor de secretos (AWS Secrets Manager / Vault).
- **DATABASE_URL:** Cadena de conexión con credenciales — nunca en código fuente ni en logs.
- **SMTP credentials:** Usuario y contraseña del proveedor de email — vía variables de entorno.
- **ACCESS_TOKEN_EXPIRE_MINUTES** y **REFRESH_TOKEN_EXPIRE_DAYS:** Configurables sin tocar código.

9 Estrategia de Pruebas

9.1 Tipos de Prueba

Tipo	Herramienta	Alcance
Unitaria	pytest + unittest.mock	Servicios y funciones de seguridad aislados (hashing, generación de tokens, validaciones).
Integración	pytest + TestClient FastAPI	Endpoints completos con BD en memoria (SQLite) o PostgreSQL de prueba.
Seguridad	OWASP ZAP (DAST)	Pruebas de penetración automatizadas: SQLi, XSS, fuerza bruta.
Carga	Locust	Simulación de 500 usuarios concurrentes en endpoints de login y refresh.
Cobertura	pytest-cov	Umbral mínimo: 80 % de cobertura de líneas. Integrado en CI/CD.

9.2 Casos de Prueba Clave

ID	Descripción	Datos de entrada	Resultado esperado	Tipo
TC-01	Registro exitoso	Email único, contraseña válida	201 Created + user_id	Éxito
TC-02	Email duplicado en registro	Email ya existente en BD	400 Bad Request	Error
TC-03	Contraseña débil en registro	Password sin mayúscula	422 Unprocessable	Error
TC-04	Login exitoso	Credenciales correctas	200 + access + refresh token	Éxito
TC-05	Login con contraseña incorrecta	Password erróneo	401 Unauthorized	Error
TC-06	Bloqueo tras 5 intentos	5 logins fallidos consecutivos	403 + tiempo de bloqueo	Seguridad
TC-07	Refresh token válido	Token no expirado y en BD	200 + nuevos tokens	Éxito
TC-08	Reutilización de refresh token	Token ya rotado	401 Unauthorized	Seguridad
TC-09	Recuperación - email existente	Email registrado	200 + email enviado	Éxito
TC-10	Recuperación - email inexistente	Email no registrado	200 (respuesta neutra)	Seguridad
TC-11	Reset con token expirado	Token > 30 min	400 Bad Request	Error

ID	Descripción	Datos de entrada	Resultado esperado	Tipo
TC-12	GET /me con token válido	Bearer token activo	200 + perfil usuario	Éxito
TC-13	GET /me sin token	Sin cabecera Authorization	401 Unauthorized	Error
TC-14	Rate limiting	> 10 req/min desde misma IP	429 Too Many Requests	Seguridad

10 Estructura del Proyecto y Despliegue

10.1 Estructura de Archivos

Ruta	Descripción
auth-service/	Raíz del microservicio
app/main.py	Punto de entrada FastAPI, registro de routers y middleware
app/api/v1/auth.py	Router con todos los endpoints de autenticación
app/core/config.py	Pydantic Settings — variables de entorno tipadas
app/core/security.py	JWT: crear/verificar tokens; bcrypt: hash/verify
app/db/session.py	Engine SQLAlchemy, SessionLocal, get_db dependency
app/models/user.py	Modelo ORM — tabla users con todos los campos
app/schemas/auth.py	Pydantic: RegisterRequest, LoginResponse, TokenResponse...
app/service-s/auth_service.py	Lógica de negocio: registro, login, refresh, reset
alembic/	Migraciones de base de datos versionadas
tests/test_register.py	Pruebas unitarias e integración: registro
tests/test_login.py	Pruebas: login, bloqueo, intentos fallidos
tests/test_refresh.py	Pruebas: rotación de tokens
tests/-test_password_reset.py	Pruebas: recuperación de contraseña
Dockerfile	Imagen multi-stage (builder + runtime)
docker-compose.yml	Servicios: auth-service + postgres + mailhog (dev)
.env.example	Plantilla de variables de entorno
requirements.txt	Dependencias: fastapi, sqlalchemy, pyjwt, passlib...

10.2 Variables de Entorno Requeridas

Variable	Valor de ejemplo	Descripción
DATABASE_URL	postgresql://user:pwd@localhost:5432/auth	Cadena de conexión PostgreSQL
SECRET_KEY	<256-bit-random-string>	Clave de firma JWT (nunca hardcodeada)
ACCESS_TOKEN_EXPIRE_MINUTES	30	Duración del access token
REFRESH_TOKEN_EXPIRE_MINUTES	7	Duración del refresh token
RESET_TOKEN_EXPIRE_MINUTES	30	Duración del token de recuperación
BCRYPT_ROUNDS	12	Cost factor de bcrypt (≥ 12 en producción)
RATE_LIMIT_PER_MINUTE	1	Solicitudes máximas por IP por minuto
SMTP_HOST / SMTP_PORT	smtp.sendgrid.net / 587	Proveedor de email para recuperación
ENVIRONMENT	production	Contexto: development staging production

11 Prompt de Ingeniería para Generación de Código con IA

El siguiente prompt estructurado debe utilizarse como entrada en herramientas de IA (Claude, ChatGPT, GitHub Copilot) para generar el microservicio completo a partir de este documento SPEC.

PROMPT ESTRUCTURADO

ROL:

Actúa como un Arquitecto de Soluciones Senior especializado en microservicios Python con FastAPI. Tu objetivo es generar código de producción, seguro, bien estructurado y con pruebas.

CONTEXTO:

Debes implementar un microservicio de autenticación (Auth Service) con Python 3.11, FastAPI, PostgreSQL 15, SQLAlchemy y PyJWT.

REQUERIMIENTOS TÉCNICOS:

- Registro de usuario: POST /api/v1/auth/register — validar email (RFC 5322), contraseña (8+ chars, mayúscula, número, especial), hashear con bcrypt cost=12.
- Login: POST /api/v1/auth/login — verificar credenciales, emitir access token JWT (15 min) y refresh token (7 días), almacenar hash del refresh token, bloqueo tras 5 intentos.
- Refresh: POST /api/v1/auth/refresh — verificar refresh token, rotación (invalidar anterior, emitir nuevo par).
- Recuperación de contraseña: POST /api/v1/auth/password-reset/request (respuesta siempre 200 OK) + POST /api/v1/auth/password-reset/confirm (token SHA-256, expiración 30 min).
- Perfil: GET /api/v1/auth/me — requiere Bearer token válido.

ENTREGABLES:

1. Código fuente completo con la estructura de archivos definida en el SPEC.
2. Suite de pruebas pytest con cobertura $\geq 80\%$.
3. Dockerfile multi-stage y docker-compose.yml.
4. Archivo .env.example con todas las variables.

RESTRICCIONES:

No uses frameworks de autenticación preconstruidos (no FastAPI-Users, no Auth0). Implementa la lógica desde cero usando las librerías indicadas. No almacenes contraseñas en texto plano bajo ninguna circunstancia.

12 Comparación de Resultados entre IAs

Esta sección debe completarse por el estudiante tras ejecutar el prompt de ingeniería en al menos dos herramientas de IA. La siguiente tabla proporciona el marco de evaluación.

Criterio de Evaluación	Claude Sonnet	ChatGPT-4o
Estructura de archivos generada	[Completar tras prueba]	[Completar tras prueba]
Implementación de hashing bcrypt	[Completar tras prueba]	[Completar tras prueba]
Implementación de JWT (access + refresh)	[Completar tras prueba]	[Completar tras prueba]
Manejo de bloqueo por intentos	[Completar tras prueba]	[Completar tras prueba]
Respuesta neutra en recuperación	[Completar tras prueba]	[Completar tras prueba]
Calidad de pruebas generadas	[Completar tras prueba]	[Completar tras prueba]
Adherencia al SPEC original	[Completar tras prueba]	[Completar tras prueba]
Código listo para producción	[Completar tras prueba]	[Completar tras prueba]
Edge cases detectados adicionales	[Completar tras prueba]	[Completar tras prueba]
Tiempo de respuesta aproximado	[Completar tras prueba]	[Completar tras prueba]

12.1 Edge Cases Identificados por IA

Durante la fase de detección de errores (Tarea 2 del enunciado), las IAs pueden identificar los siguientes edge cases no cubiertos en el SPEC inicial:

#	Edge Case	Descripción / Riesgo	Severidad
1	Timing attack en comparación de tokens	Comparar tokens con == en Python puede filtrar información por tiempo de ejecución. Usar hmac.compare_digest().	Alta
2	Email con caracteres Unicode	Emails con caracteres Unicode pueden bypasear validaciones regex simples.	Media
3	Contraseña con espacios	La política no especifica si se permiten espacios. Puede causar inconsistencias.	Baja
4	Refresh token en múltiples dispositivos	El SPEC invalida todos los refresh tokens al rotar, afectando sesiones paralelas.	Media

#	Edge Case	Descripción / Riesgo	Severidad
5	Race condition en registro	Dos solicitudes concurrentes con el mismo email pueden pasar la verificación de unicidad.	Alta
6	Expiración del access token durante request	El token puede expirar entre la validación y el procesamiento de la solicitud.	Baja
7	Tamaño máximo de campos	El SPEC no define límite en el nombre completo, susceptible a payload grande.	Baja

13 Glosario

Término	Definición
Access Token	Token JWT de corta duración (15 min) que autoriza el acceso a recursos protegidos.
bcrypt	Algoritmo de hashing adaptativo diseñado para contraseñas, resistente a ataques de fuerza bruta.
Bearer Token	Esquema de autenticación HTTP donde el token se envía en la cabecera Authorization.
CORS	Cross-Origin Resource Sharing — política que controla qué orígenes pueden acceder a la API.
FastAPI	Framework web Python moderno, basado en Starlette y Pydantic, con generación automática de OpenAPI.
JWT	JSON Web Token — estándar RFC 7519 para transmitir información de forma segura entre partes como JSON firmado.
Microservicio	Servicio autónomo, con responsabilidad única, desplegable y escalable de forma independiente.
OWASP	Open Web Application Security Project — organización que publica estándares de seguridad web.
Pydantic	Librería Python para validación de datos y gestión de configuración mediante type hints.
Refresh Token	Token JWT de larga duración (7 días) usado exclusivamente para obtener nuevos access tokens.
RFC 5322	Estándar que define el formato de mensajes de email, incluida la sintaxis de direcciones.
Rotación de Tokens	Práctica de invalidar el refresh token al usarlo y emitir uno nuevo, limitando ventanas de ataque.
SQLAlchemy	ORM (Object-Relational Mapper) para Python que abstrae operaciones con bases de datos relacionales.
TLS	Transport Layer Security — protocolo criptográfico que asegura las comunicaciones en red.

— Fin del Documento —